Design Pattern Remote Proxy e Forwarder-Receiver

Design Pattern Remote Proxy

Intento e contesto

Il Remote Proxy è una variante del pattern Proxy utilizzata quando un client necessita di accedere ai servizi di un oggetto (componente) che risiede su un processo o host diverso (cioè su un altro spazio di indirizzamento). In questo contesto stiamo quindi parlando di comunicazione di rete.

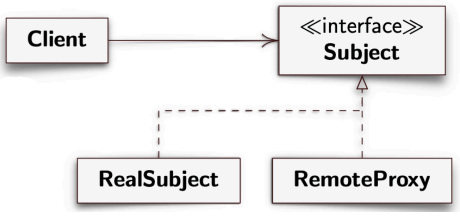
Il **problema** è che l'accesso diretto ad un oggetto remoto presenta diverse criticità:

1. **Complessità**: il client non dovrebbe conoscere dettagli di basso livello come indirizzi IP, porte o protocolli di comunicazione inter-processo (IPC).
2. **Trasparenza**: l'accesso al servizio remoto dovrebbe apparire al client semplice come una chiamata locale. Il client non deve cambiare sintassi.
3. **Prestazioni**: sebbene l'accesso debba essere sintatticamente trasparente, il client deve essere consapevole che una chiamata remota è molto più lenta di una chiamata locale (ad esempio, 20.000 ns contro 1 ns). Questo può dipendere dalle distanze, velocità di rete e tecnologie utilizzate.

La **soluzione** al problema è l'introduzione di un oggetto RemoteProxy che agisce come rappresentante locale dell'oggetto remoto (RealSubject).

- Il client comunica solo con il RemoteProxy, che manda richieste ad un host remoto, dove il RealSubject risiede, ricevendo quindi risposte da remoto.
- Il RemoteProxy si occupa di inoltrare la richiesta attraverso la rete all'host dove risiede il RealSubject e di ricevere la risposta.
- Il Proxy nasconde la posizione (indirizzo IP, porta) e implementa il protocollo di comunicazione (utilizza le socket per stabilire la comunicazione tra le due macchine e i due processi).
- Per meccanismi IPC complessi si può usare un componente specifico lato client, e analogamente un componente ricevente per il lato RealSubject. I client non sono a conoscenza dei dettagli della comunicazione remota.
- Il RemoteProxy può far comunicare due processi sullo stesso host o su host diversi.
- Il RemoteProxy rende più facile analizzare le prestazioni (e anche i ritardi) della comunicazione in rete.

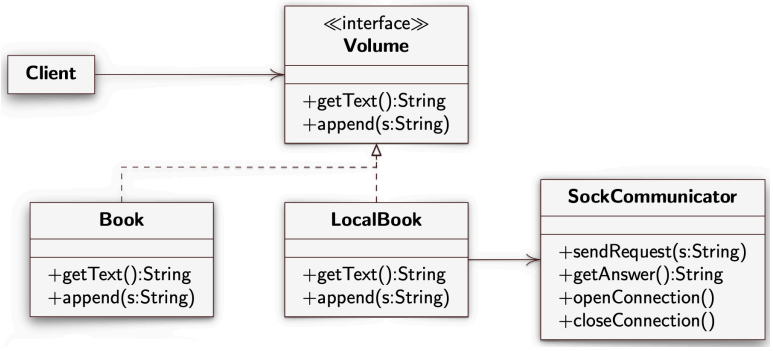
Diagramma UML delle classi



Struttura e partecipanti

Prendiamo l'esempio basato su un'interfaccia Volume (Subject) e una classe Book (RealSubject):

1. **Client**: chiama i metodi sull'interfaccia Volume aspettandosi un comportamento locale. Non sa se l'oggetto è locale o remoto.
2. **Volume** (Subject): l'interfaccia comune che definisce i metodi (es. getText(), append()). Serve per garantire trasparenza.
3. **Book** (RealSubject): l'oggetto vero e proprio che risiede sul server, che contiene lo stato e la logica reale.
4. **RemoteProxy** (LocalBook): risiede sul client, traduce le chiamate ai metodi in messaggi di rete (inviati tramite SockCommunicator) e gestisce la comunicazione (implementando un protocollo apposito con l'altra parte). Una volta ricevuta la risposta dal server, il RemoteProxy restituisce il risultato al client, a cui tutto è trasparente.



Helper di comunicazione

- **SockCommunicator**: un componente lato client che astrae l'uso delle socket, inviando stringhe di richiesta e ricevendo risposte. Questo nasconde la gestione TCP.
- **SockListener**: un componente lato server che ascolta i messaggi in arrivo dalla rete, inoltrandoli quindi al ServerProxy.
- **ServerProxy**: proxy lato server, che traduce i messaggi ricevuti in chiamate reali all'oggetto Book, prende il risultato e lo rimanda indietro.

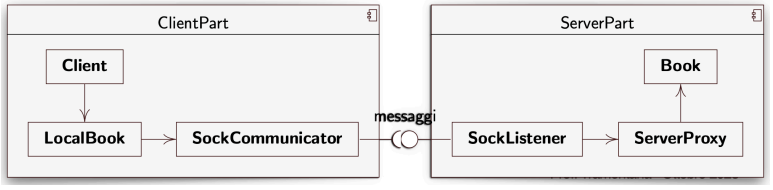


Diagramma UML di sequenza Remote Proxy (Client-Side)

- Il Client invoca getText() su LocalBook (Proxy).
- LocalBook usa SockCommunicator per aprire la connessione (openConnection()) e inviare la richiesta (sendRequest()).
- Il Client attende la risposta (getAnswer()).

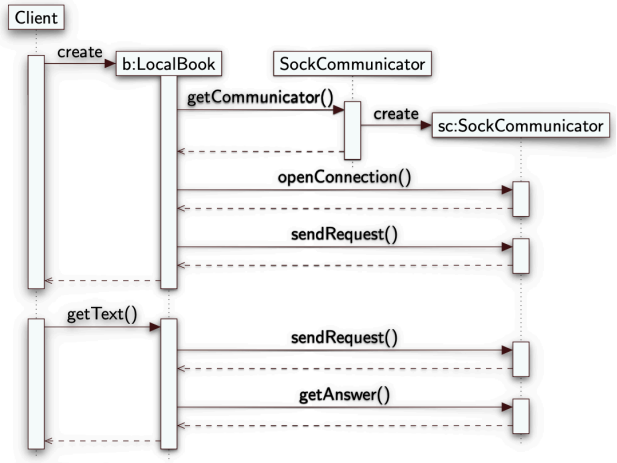
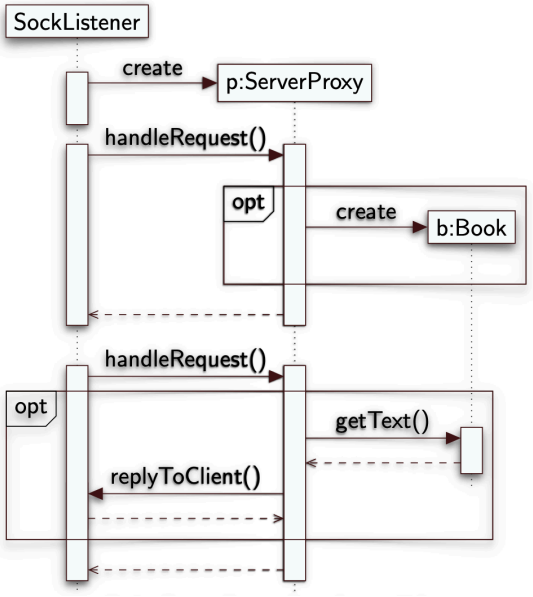


Diagramma UML di sequenza Remote Proxy (Server-Side)

- SockListener riceve la richiesta di rete.
- Passa la richiesta al ServerProxy (tramite handleRequest()).
- Il ServerProxy interpreta il messaggio, invoca il metodo reale getText() sull'oggetto Book.
- Il risultato viene catturato e inviato indietro al client (replyToClient()).



Design Pattern Forwarder-Receiver

Intento e contesto

Il pattern Forwarder-Receiver è un pattern architetturale progettato per sistemi che interagiscono secondo un modello peer-to-peer. Il suo intento principale è quello di fornire una comunicazione trasparente tra processi, disaccoppiando i componenti software (cioè i peer) dai meccanismi sottostanti di comunicazione di rete (come socket, TCP/IP o code di messaggi). Quindi, l'applicazione non deve sapere come i messaggi viaggiano.

Il **problema** è che quando si sviluppano applicazioni distribuite, l'uso diretto di meccanismi di basso livello (comunicazioni inter-processo IPC) come le socket crea forti dipendenze tra il codice applicativo e il sistema operativo o i protocolli di rete. Questo comporta due svantaggi principali:

1. È difficile cambiare il meccanismo di comunicazione in futuro senza riscrivere l'applicazione.
2. I componenti devono conoscere dettagli fisici (indirizzi IP, porte) invece di comunicare tramite nomi logici.

La **soluzione** al problema è data dall'introduzione di due componenti intermedi, il **Forwarder** (chi spedisce) e il **Receiver** (chi riceve), per gestire tutto il traffico di rete, lasciando i componenti applicativi (Peer) completamente ignari dei dettagli di rete.

I partecipanti sono tre:

1. **Peer**: è il componente dell'applicazione (che può agire sia da client che da server nel modello peer-to-peer). Svolge i compiti di business e deve inviare o ricevere messaggi. Il Peer non conosce i dettagli della rete, quindi non sa nulla di socket, protocolli o indirizzi IP. Comunica usando soltanto nomi logici, non indirizzi fisici.
2. **Forwarder**: è responsabile dell'invio dei messaggi.
   - fornisce l'interfaccia pubblica per inviare i dati.
   - mappa i nomi logici dei destinatari in indirizzi fisici (locazioni).
   - serializza i dati e li spedisce attraverso il canale di comunicazione IPC.
   - include nel messaggio il nome del peer mittente per permettere la risposta.
3. **Receiver**: è responsabile della ricezione dei messaggi.
   - offre un'interfaccia per il meccanismo di comunicazione specifico (es. si mette in ascolto su una socket).
   - converte il flusso (stream) di dati in arrivo in un formato comprensibile per l'applicazione (messaggio) e lo inoltra quindi al Peer corretto.

Tipi di messaggi

Il pattern gestisce diverse tipologie di messaggi scambiati tra i Peer:

- **Messaggi comando**: istruiscono il ricevente a svolgere una determinata attività.
- **Messaggi informativi**: contengono dati sullo stato delle risorse o su eventi di rete.
- **Messaggi di risposta**: conferme (ack) per un messaggio ricevuto o risultati di operazioni.

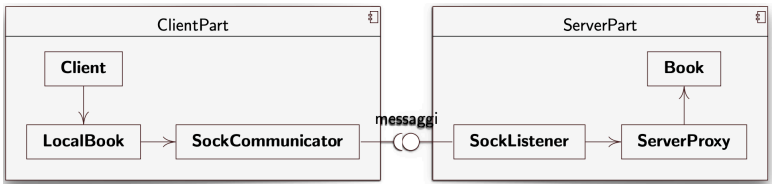
Relazione con il Remote Proxy

Il Forwarder-Receiver è un supporto all'implementazione del Remote Proxy. Mentre il Remote Proxy si occupa di fornire l'interfaccia dell'oggetto remoto (cioè far sembrare locale una chiamata in realtà remota), il Forwarder-Receiver gestisce il "come" i dati viaggiano (trasporto del messaggio vero e proprio).

<u>Esempio: sistema Client-Server per un libro</u>

- **Peer**: le classi *LocalBook* (il *Remote Proxy lato client*) e *ServerProxy* (*Proxy lato server*) agiscono come Peer che devono parlarsi.
- **Forwarder**: la classe *SockCommunicator* astrae l'invio tramite socket.

- **Receiver**: la classe *SockListener* riceve i messaggi dalla rete.



L'uso combinato dei due pattern permette al RemoteProxy di conoscere solo il nome logico del destinatario, lasciando al Forwarder la responsabilità di sapere dove si trova fisicamente (indirizzo IP / porta).